

Eventia

Resumen del proyecto y del módulo de autenticación

Grupo 1 - Ingeniería de Software II

Fecha: 26 de agosto de 2026

1. Enunciado del proyecto

La organización de eventos académicos, profesionales o institucionales requiere gestionar participantes, conferencistas, actividades y agenda del evento. El sistema busca centralizar estas tareas para mejorar el control de inscripciones, la planificación y el registro de asistencia.

El sistema deberá permitir:

- Crear y administrar eventos.
- Gestionar participantes y conferencistas.
- Organizar la agenda de actividades.
- Registrar la asistencia a sesiones o actividades.
- Administrar tipos de eventos como congresos, seminarios, talleres y jornadas.
- Gestionar inscripciones, cupos y listas de participantes.
- Generar certificados o registros de participación.
- Configurar tipos de eventos, categorías, roles, políticas de inscripción, límites de cupo y formatos de agenda.

2. Modelo entidad-relación

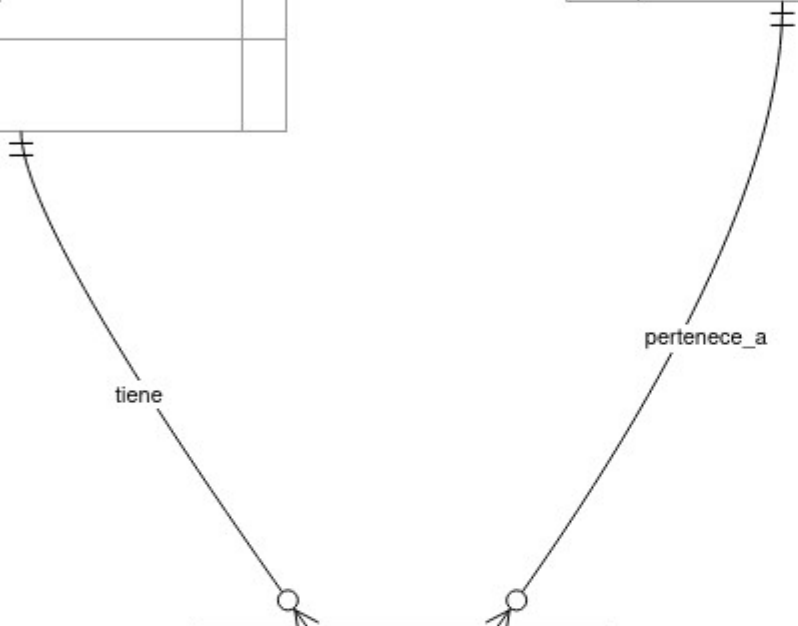
El modelo presentado contiene las entidades principales del dominio:

- `USUARIO`, `ROL_SISTEMA` y `USUARIO_ROL` para usuarios y roles.
- `TIPO_EVENTO` y `EVENTO` para la administración de eventos.
- `PARTICIPANTE` y `ROL_PARTICIPANTE` para las personas inscriptas.
- `ACTIVIDAD`, `CATEGORIA_ACTIVIDAD`, `CONFERENCISTA` y `ACTIVIDAD_CONFERENCISTA` para la agenda.
- `POLITICA_INSCRIPCION` e `INSCRIPCION` para las reglas de registro.
- `ASISTENCIA` y `CERTIFICADO` para participación y acreditación.

USUARIO		
int	id_usuario	PK
string	nombres	
string	apellidos	
string	email	
string	password_hash	
boolean	estado	

ROL_SISTEMA		
int	id_rol	PK
string	nombre	
string	descripcion	

USUARIO_ROL		
int	id_usuario	PK, FK
int	id_rol	PK, FK



define

Diagrama entidad-relación utilizado como referencia del proyecto.

3. Módulo: Gestión de Usuarios, Roles y Seguridad

El módulo contempla la administración de usuarios, la asignación de roles del sistema y el control de acceso a las funcionalidades. Las entidades centrales son:

- **USUARIO**: identidad, correo, estado y hash de contraseña.
- **ROL_SISTEMA**: roles disponibles y su estado.
- **USUARIO_ROL**: relación muchos a muchos entre usuarios y roles.

Como funcionalidades complementarias se identificaron el alta, edición y desactivación de usuarios, el inicio y cierre de sesión, la recuperación de contraseña y la autorización por roles.

4. Historias de usuario

4.1 Autenticación de usuarios

Descripción: Como usuario del sistema, quiero iniciar sesión con mi correo y contraseña para acceder a las funcionalidades correspondientes a mi perfil.

La implementación utiliza el endpoint `POST /api/v1/auth/login`, valida las credenciales contra el hash seguro de contraseña y genera un token JWT.

Criterios de aceptación:

- Dadas credenciales válidas, el sistema compara la contraseña ingresada con `password_hash` mediante Argon2, verifica que el usuario esté activo y otorga acceso con un token de sesión JWT.
- Dadas credenciales incorrectas, el sistema responde con `401 Credenciales inválidas` sin especificar si falló el correo o la contraseña.

4.2 Middleware y tokens de sesión

Configurar la validación de tokens JWT para proteger los endpoints de la API.

- Generar un JWT con el identificador del usuario y sus roles.
- Definir una fecha de expiración.
- Leer el encabezado `Authorization: Bearer TOKEN`.
- Rechazar tokens ausentes, inválidos o vencidos.

- Permitir autorización basada en roles.

4.3 Parametrización de roles de sistema y participantes

Como administrador, se pueden crear y administrar roles de usuario y tipos de participantes para controlar la seguridad y clasificar a los asistentes a un evento.

- Los roles de sistema y de participantes se administran como catálogos parametrizables.
- Los nombres son únicos, sin distinguir mayúsculas de minúsculas.
- Las operaciones administrativas requieren un JWT con rol `administrador`.
- La eliminación de un rol de participante se bloquea con conflicto `409` si existen inscripciones activas asociadas.
- La eliminación de un rol de sistema se bloquea si está asignado a usuarios.

```
GET    /api/v1/roles/sistema
POST   /api/v1/roles/sistema
PUT    /api/v1/roles/sistema/{role_id}
DELETE /api/v1/roles/sistema/{role_id}

GET    /api/v1/roles/participantes
POST   /api/v1/roles/participantes
PUT    /api/v1/roles/participantes/{role_id}
DELETE /api/v1/roles/participantes/{role_id}
```

La API conserva PATCH como alias de actualización por compatibilidad.

5. Implementación realizada

Se preparó un backend REST con Python, FastAPI, SQLAlchemy y PostgreSQL:

```
backend/
├─ app/
│  ├─ api/
│  │  ├─ auth.py          # POST /api/v1/auth/login
│  │  ├─ roles.py         # Parametrización de roles
│  │  └─ users.py         # GET /api/v1/users/me
│  ├─ core/
│  │  ├─ config.py        # Variables de entorno
│  │  └─ security.py      # Argon2 y JWT
│  ├─ db.py               # Conexión SQLAlchemy
│  ├─ models.py           # Usuario y roles
│  ├─ schemas.py          # Solicitudes y respuestas
│  └─ main.py             # Aplicación FastAPI
├─ .env.example
└─ requirements.txt
```

El login devuelve el token, su tipo, el tiempo de expiración y los datos públicos del usuario. Nunca devuelve `password_hash`. La parametrización usa `ROL_SISTEMA`, `ROL_PARTICIPANTE`, `USUARIO_ROL` e `INSCRIPCION`.

6. Ejecución local

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload
```

Configurar en `.env` la conexión a PostgreSQL:

```
DATABASE_URL=postgresql+psycopg://usuario:password@localhost:5432/eventia
JWT_SECRET_KEY=una-clave-secreta-de-al-menos-32-caracteres
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=60
```

La documentación interactiva está disponible en `http://127.0.0.1:8000/docs`. También existe el endpoint `GET /health` para comprobar que el servidor está activo.

7. Validaciones realizadas

- Compilación de todos los módulos Python mediante `compileall`.
- Registro correcto de las rutas de login y perfil protegido en OpenAPI.
- Hash y verificación de contraseñas mediante Argon2.
- Generación de JWT con roles y expiración.
- Pruebas de autenticación válida, contraseña incorrecta y correo inexistente mediante `tests/test_auth.py`.
- Pruebas de integridad de roles mediante `tests/test_role_integrity.py`.

La prueba contra una base PostgreSQL real requiere configurar el servidor y crear las tablas del modelo.